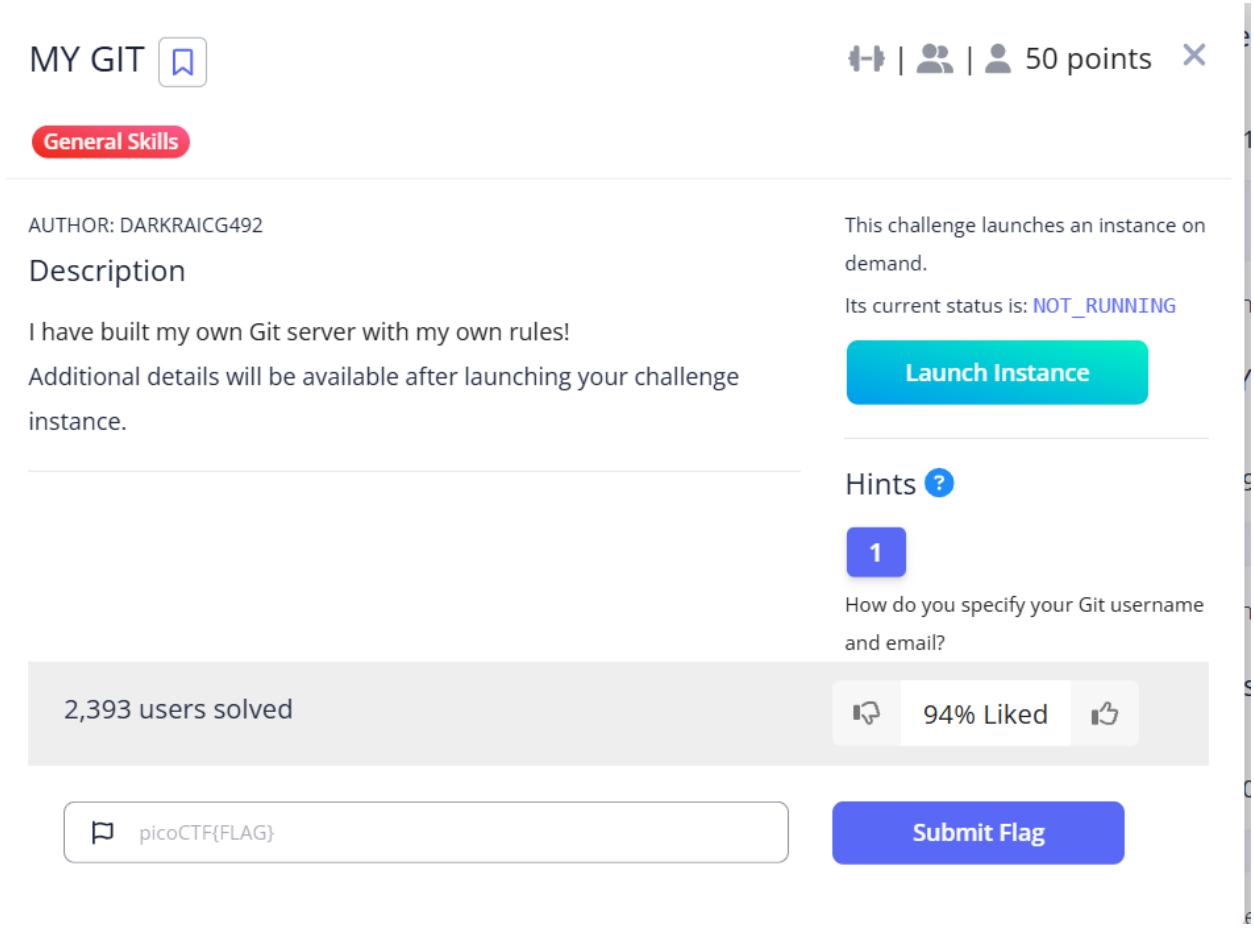
Saint Columban College
College of Computing Studies
Pagadian City

MY GIT



Author: The Analyst: Hyposelenia

Challenge: MY GIT

Category: General Skills

Date: 03/16/26

I. Objective

The objective of this challenge is to interact with a remote Git repository, modify the Git configuration to match a specific identity, and successfully push a commit in order to retrieve the hidden flag.

II. Background

Git is a version control system used by programmers to track changes in their code. It records every modification made to files and allows developers to collaborate by sharing repositories.

Each change made in Git is called a **commit**, which includes information about:

- who made the change
- the email of the author
- what files were changed
- a message describing the update

This identity information is stored in **Git configuration settings** such as:

- user.name
- user.email

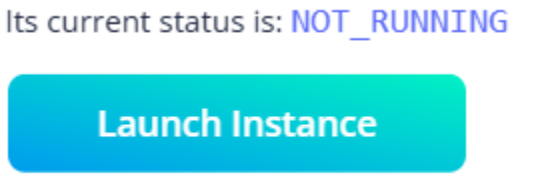
In this challenge, the repository required the correct identity before allowing the user to push a commit. By configuring Git with the correct username and email, the repository accepts the change and reveals the flag.

III. Tool Used

• **Oracle VirtualBox (Kali Linux)** – Used as the Linux environment
• **Linux Terminal** – Used to execute commands and interact with Git
• **Git** – Version control tool used to manage and push changes to the repository

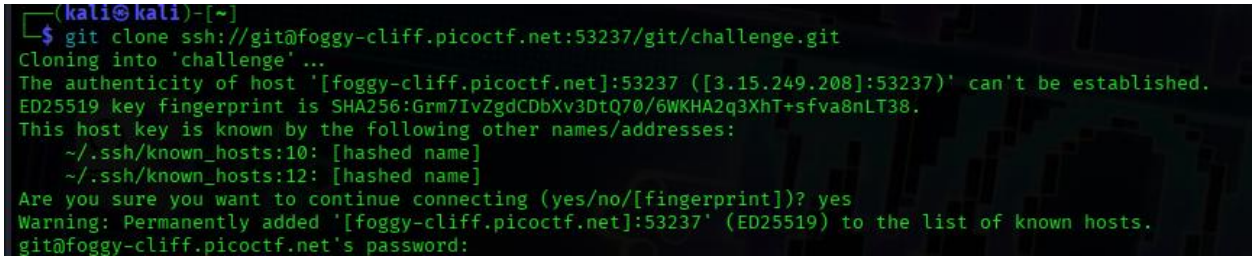
IV. Methodology

1. Launched **Kali Linux** using VirtualBox.
2. Launched the **challenge instance**.

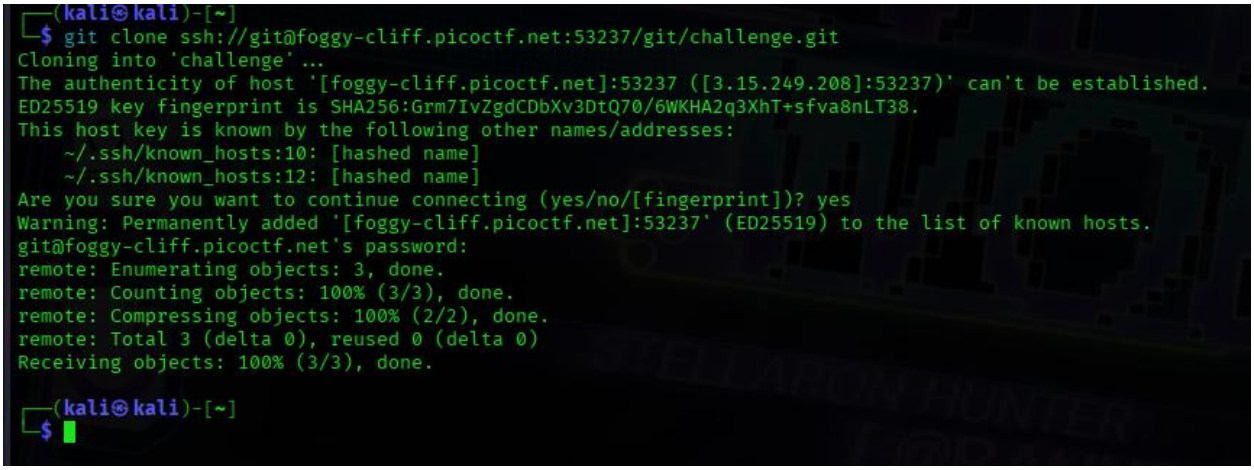


3. Typed the command provided in the challenge description to connect to the repository.

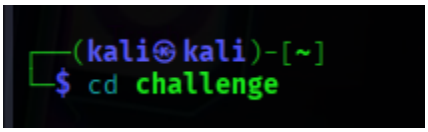
4. Confirmed the connection by typing **yes** when prompted.



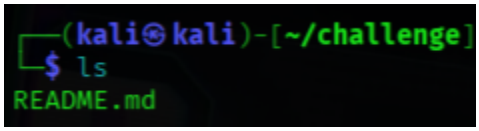
5. Entered the provided password to authenticate. (you can't see it)



6. Changed the working directory to the repository folder using:
   cd challenge



7. Listed the files inside the directory using:
   ls



8. Observed that the challenge folder contained a file named **README.md**.
9. Displayed the contents of the file using:
   cat README.md

```
┌──(kali㉿kali)-[~/challenge]
└─$ cat README.md
# MyGit

### If you want the flag, make sure to push the flag!

Only flag.txt pushed by ```root:root@picoctf``` will be updated with the flag.

GOOD LUCK!
```

10. Observed that the contents contained in the file named **README.md**, I found the author's username and email.

```
```root:root@picoctf```
```

11. Displayed the contents of the file using:
git config user.name "username"

```
┌──(kali㉿kali)-[~/challenge]
└─$ git config user.name "root"
```

12. Configured the Git email:
git config user.email "email"

```
┌──(kali㉿kali)-[~/challenge]
└─$ git config user.email "root@picoctf"
```

13. Verified that the configuration was correctly applied using:
git config –list
```

```
┌──(kali㉿kali)-[~/challenge]
└─$ git config --list
safe.directory=/media/sf_Downloaded_Cybersecfiles/drop-in
user.email=hyposeleniacutie@gmail.com
user.name=Hyposelenia
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
remote.origin.url=ssh://git@foggy-cliff.picoctf.net:53237/git/challenge.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
branch.master.remote=origin
branch.master.merge=refs/heads/master
user.name=root
user.email=root@picoctf
```

14. Created a new file and added content to it.

```
┌──(kali㉿kali)-[~/challenge]
└─$ echo "flag please, I order you to bring it before me" > flag.txt
```

15. Added the file to the Git staging area:

git add file.txt

```
┌──(kali㉿kali)-[~/challenge]
└─$ git add flag.txt
```

16. Created a commit with a message:

git commit -m "Added new file"

```
┌──(kali㉿kali)-[~/challenge]
└─$ git commit -m "Added flag file"
[master 6dbc511] Added flag file
 1 file changed, 1 insertion(+)
 create mode 100644 flag.txt
```

17. Pushed the commit to the remote repository:

git push origin master

```
┌──(kali㉿kali)-[~/challenge]
└─$ git push origin master
git@foggy-cliff.picoctf.net's password:
```

18. Entered the password again when prompted.

Here's the password: d3d09b5e

19. After the push was accepted, the **flag was displayed**.

```
┌──(kali㉿kali)-[~/challenge]
└─$ git push origin master
git@foggy-cliff.picoctf.net's password:
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 314 bytes | 314.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Author matched and flag.txt found in commit ...
remote: Congratulations! You have successfully impersonated the root user
remote: Here's your flag: picoCTF{1mp3rs0n4t4_g17_345y_6877715a}
To ssh://foggy-cliff.picoctf.net:53237/git/challenge.git
   2001d2f..6dbc511  master → master
```

V. Result

picoCTF{1mp3rs0n4t4_g17_345y_6877715a}

```
remote: Congratulations! You have successfully impersonated the root
remote: Here's your flag: picoCTF{1mp3rs0n4t4_g17_345y_6877715a}
To ssh://foggy-cliff.picoctf.net:53237/git/challenge.git
```

VI. Explanation

Think of **Git** like a **school notebook for programmers**.

Every time someone writes something new in the notebook, they sign their name so everyone knows **who wrote it**. Git does the same thing.

When someone makes a change, Git records:

- the **name of the person**
- the **email**
- what they changed

These are called **Git configurations**.

What each command does?

git

Git is a tool that remembers every change made to files. It is like a **history book for code**.

What's a git config?
This command tells Git **who you are**.

Example:
git config user.name "username"
This sets the **name of the person making the change**.
git config user.email "email"
This sets the **email address of the person making the change**.

What's a git config –list?
This command shows **all the Git settings currently saved**.
It helps confirm that the correct identity was applied.

What's a README.md?

This is a common file found in many repositories. README means **"Read Me"**.

It usually contains:

- instructions
- notes about the project
- useful information

In this challenge, it contained the **identity we needed to impersonate**.

What's a git add?

git add file.txt

This tells Git:

"I want this file to be included in the next update."

It moves the file into a place called the **staging area**.

What's a git commit?

git commit -m "message"

This saves the change into Git's history.

The **message** explains what change was made.

Example:

"Added new file"

What's a git push?

git push origin master

This sends the changes from the local computer **to the remote repository**. Think of it like **uploading your homework to the teacher's system**.

Why the challenge worked?

The repository expected commits from a **specific identity**.

By changing the Git configuration to match the author mentioned in the README file, we **impersonated the correct user**.

Because the identity matched, the repository accepted the commit and returned the flag.

VII. Conclusion

The MY GIT challenge demonstrates how Git stores user identity information within commits and how this identity can be configured or modified. By analyzing the repository instructions and setting the correct Git username and email, a valid commit was successfully pushed to the repository. This allowed the system to accept the change and reveal the hidden flag, highlighting how identity configuration plays an important role in version control systems.

— **The Analyst: Hyposelenia**